

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED: CO-2: Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool: Design Task

Weightage: 20%

Total Marks: 20

STUDENT DETAILS

Name:	Sameera Banu. P
Register Number:	192372172
Course Name:	Internet Programming
Course Code:	CSA43
Assessment Tool:	Assessment Tool 2 – Design Task
Course Outcome:	CO2

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Identify key components required in a navigation bar for an e-commerce website.

A comprehensive e-commerce navigation bar serves as the primary visual portal and user routing control center. To maximize user conversion and seamless browsing, the following key components are required:

- **Brand Logo & Identity Header:** Placed at the top-left section, serving as brand anchor and immediate direct hyperlink to the store's homepage.
- **Categorical Navigation Links:** Primary links providing access to main store departments (e.g., Home, Shop, Categories, Deals, Contact).

- **Product Search Bar:** A prominent input field with a search icon/button enabling users to query the inventory directly.
- **User Utility & Cart Module:** Quick access icons for user account settings, wishlist, and shopping cart equipped with a dynamic item counter badge.
- **Responsive Hamburger Menu Button:** A compact three-bar icon displayed on mobile screen sizes to open and close collapsible vertical navigation drawers.

(b) Design a responsive navigation bar using appropriate CSS techniques.

(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).

Short Explanation:

The solution utilizes CSS Flexbox for horizontal alignment across desktop viewports. Media queries (`@media (max-width: 768px)`) dynamically transform the horizontal bar into a collapsible vertical menu on mobile screens. JavaScript handles the click event on the hamburger icon, toggling an `active` CSS class to reveal or hide navigation links smoothly.

Complete Source Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>E-Commerce Responsive Navigation Bar</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family:
'Times New Roman', Times, serif; }
    body { background-color: #f4f6f9; }

    /* Navigation Container */
    .navbar {
      display: flex;
      justify-content: space-between;
      align-items: center;
      background-color: #1a252f;
      padding: 14px 28px;
      color: #ffffff;
      position: relative;
      box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    }

    .logo { font-size: 22px; font-weight: bold; color: #00bcd4; text-
decoration: none; }

    /* Search Box */
    .search-box { display: flex; align-items: center; flex: 0 1 280px;
margin: 0 15px; }
    .search-box input {
      width: 100%; padding: 7px 10px; border: 1px solid #ccc; border-
radius: 4px 0 0 4px; outline: none;
    }
    .search-box button {
      padding: 7px 12px; background-color: #00bcd4; border: none;
color: #ffffff;
      border-radius: 0 4px 4px 0; cursor: pointer; font-weight: bold;
    }

    /* Navigation Links */
    .nav-links { display: flex; list-style: none; gap: 20px; align-
items: center; }
    .nav-links a { color: #ffffff; text-decoration: none; font-size:
15px; transition: color 0.3s; }
    .nav-links a:hover { color: #00bcd4; }

    /* Cart Badge */
    .cart-badge { background-color: #e74c3c; color: #ffffff; padding:
2px 6px; border-radius: 10px; font-size: 11px; margin-left: 4px; }

```

```

    /* Hamburger Menu Toggle */
    .hamburger { display: none; flex-direction: column; cursor:
pointer; gap: 4px; }
    .hamburger span { width: 24px; height: 3px; background-color:
#ffffff; border-radius: 2px; }

    /* Tablet and Mobile Screen Styles */
    @media (max-width: 768px) {
        .search-box { display: none; }
        .hamburger { display: flex; }
        .nav-links {
            position: absolute; top: 100%; left: 0; width: 100%;
            background-color: #1a252f; flex-direction: column;
            align-items: center; padding: 18px 0; gap: 14px; display: none;
            box-shadow: 0 4px 8px rgba(0,0,0,0.2);
        }
        .nav-links.active { display: flex; }
    }
</style>
</head>
<body>

<nav class="navbar">
    <a href="#" class="logo">E-Store</a>
    <div class="search-box">
        <input type="text" placeholder="Search product...">
        <button type="button">Search</button>
    </div>
    <ul class="nav-links" id="navLinks">
        <li><a href="#">Home</a></li>
        <li><a href="#">Shop</a></li>
        <li><a href="#">Categories</a></li>
        <li><a href="#">Deals</a></li>
        <li><a href="#">Cart <span class="cart-badge">3</span></a></li>
    </ul>
    <div class="hamburger" id="hamburger" onclick="toggleMenu()">
        <span></span><span></span><span></span>
    </div>
</nav>

<script>
    function toggleMenu() {
        document.getElementById('navLinks').classList.toggle('active');
    }
</script>
</body>
</html>

```

Output & Browser Screenshots:

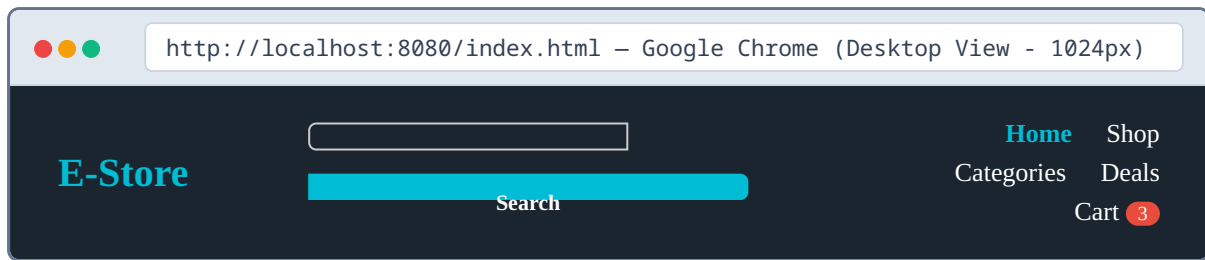


Figure 1: Responsive Navigation Bar

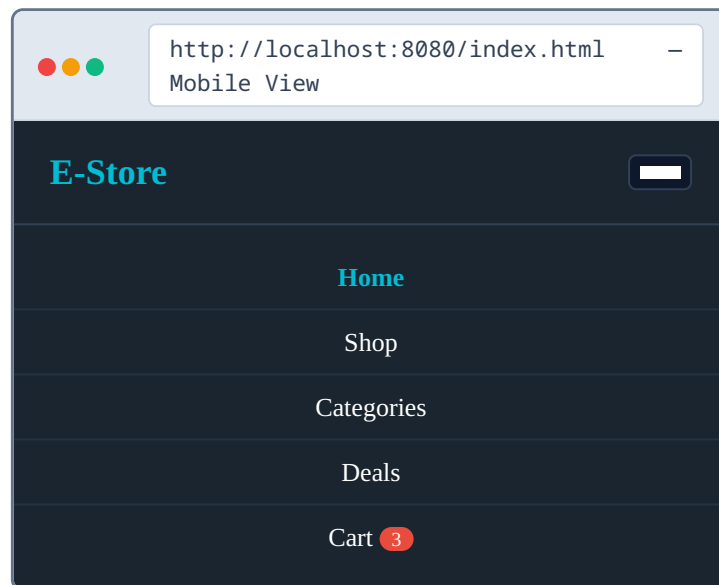


Figure 2: Mobile Navigation

(d) Evaluate the usability and suggest improvements for better user experience.

Usability Evaluation: The Flexbox layout maintains consistent spatial distribution across high-resolution desktop screens, while the mobile drawer hides dense navigational options, keeping the interface uncluttered on mobile devices.

UX & Accessibility Improvements:

- **ARIA Accessibility:** Add `aria-expanded="false"` and `aria-label="Toggle navigation menu"` to the hamburger toggle for screen reader compatibility.
- **Sticky Navigation:** Apply `position: sticky; top: 0; z-index: 1000;` to keep navigation accessible during vertical page scrolling.
- **Mobile Search Modal:** Replace the hidden search bar on mobile screens with a search icon that triggers a full-width popup search modal.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects.

Interactive CSS hover effects improve affordance and visual engagement. Key CSS properties include:

- `transform` : Executes smooth geometric shifts such as elevation (`translateY(-8px)`) and scaling (`scale(1.05)`).
- `transition` : Specifies the duration, timing curve (e.g., `ease-in-out`), and delay for property state changes.
- `box-shadow` : Increases drop-shadow blur and spread to simulate elevation in 3D space on hover.
- `opacity` : Controls smooth fading of hidden overlay controls, such as "Quick View" or "Add to Cart" buttons.
- `filter` : Applies dynamic visual filters like brightness or blur adjustment to featured product imagery.

(b) Suggest appropriate CSS animation or transition for product interaction.

Recommendation: CSS Transitions using GPU-accelerated properties (`transform` and `opacity`) are optimal. Using a cubic-bezier timing function (`transition: transform 0.3s cubic-bezier(0.25, 0.8, 0.25, 1), box-shadow 0.3s ease`) produces smooth, 60fps animations. This avoids trigger layout reflows associated with animating dimensions like `height` or `margin` .

(c) Design a hover effect for a product card.

Short Explanation:

The product card elevates upward on hover via `transform: translateY(-8px)` , accompanied by a soft drop shadow expansion. Simultaneously, the product icon scales slightly and an "Add to Cart" button slides up into view from the bottom overlay container.

Complete Source Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Product Card Hover Effect</title>
  <style>
    body {
      font-family: 'Times New Roman', Times, serif;
      background-color: #f1f5f9;
      display: flex; justify-content: center; align-items: center;
      min-height: 100vh; margin: 0;
    }

    /* Product Card Container */
    .product-card {
      width: 260px; background: #ffffff; border-radius: 10px;
      overflow: hidden; box-shadow: 0 4px 10px rgba(0, 0, 0, 0.08);
      transition: transform 0.3s ease, box-shadow 0.3s ease;
      cursor: pointer; position: relative;
    }

    /* Card Elevation on Hover */
    .product-card:hover {
      transform: translateY(-8px);
      box-shadow: 0 12px 22px rgba(0, 0, 0, 0.18);
    }

    .image-box {
      width: 100%; height: 180px; background-color: #e2e8f0;
      display: flex; justify-content: center; align-items: center;
      position: relative; overflow: hidden;
    }

    .product-icon { font-size: 56px; transition: transform 0.4s ease; }
    .product-card:hover .product-icon { transform: scale(1.12); }

    /* Action Button Overlay */
    .cart-btn {
      position: absolute; bottom: 12px; left: 50%;
      transform: translateX(-50%) translateY(30px);
      background-color: #00bcd4; color: #ffffff;
      padding: 8px 16px; border: none; border-radius: 20px;
      font-weight: bold; opacity: 0; transition: all 0.3s ease;
    }

    .product-card:hover .cart-btn {
      opacity: 1; transform: translateX(-50%) translateY(0);
    }
  </style>
</head>
</html>

```

```

        .card-body { padding: 15px; }
        .category { font-size: 11px; color: #64748b; text-transform:
uppercase; }
        .title { font-size: 16px; font-weight: bold; margin: 5px 0; color:
#1e293b; }
        .price { font-size: 15px; color: #e74c3c; font-weight: bold; }
    </style>
</head>
<body>

    <div class="product-card">
        <div class="image-box">
            <div class="product-icon"><img alt="Headphones icon" data-bbox="465 278 488 295"/></div>
            <button class="cart-btn">Add to Cart</button>
        </div>
        <div class="card-body">
            <div class="category">Electronics</div>
            <div class="title">Wireless Headphones</div>
            <div class="price">$129.99</div>
        </div>
    </div>

</body>
</html>

```

Output & Browser Screenshots:

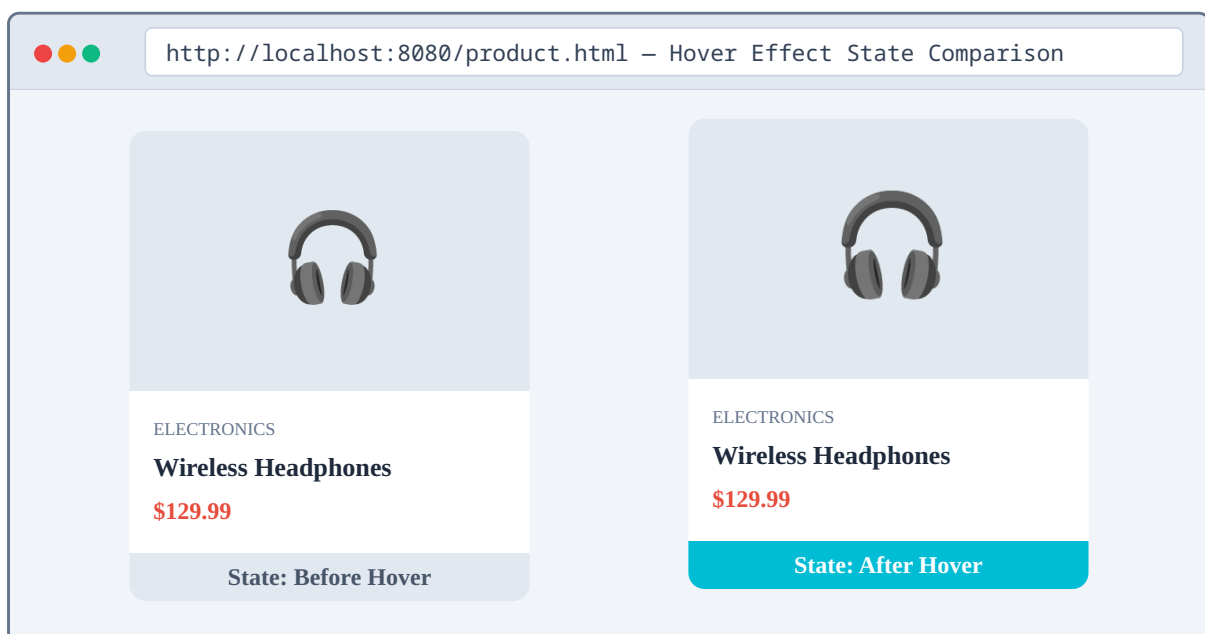


Figure 3: Product Hover Effect

(d) Evaluate performance and user experience considerations.

Performance Considerations: Animating composite properties (`transform` and `opacity`) offloads rendering to the GPU, avoiding expensive layout recalculations and repaints.

User Experience Considerations:

- **Touch Device Compatibility:** Hover effects are unavailable on mobile touchscreens. Essential functionality like "Add to Cart" should remain accessible without requiring hover.
- **Accessibility (Reduced Motion):** Include a `@media (prefers-reduced-motion: reduce)` query to disable intensive animations for users sensitive to motion.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

An enterprise dashboard requires a structured two-dimensional architecture comprising:

- **Header Bar (Row 1):** Full-width container holding titles, search input, and profile details.
- **Sidebar Panel (Column 1):** Vertical navigation column for application routing.
- **KPI Summary Grid (Multi-Column Row):** Responsive cards displaying key metrics with consistent alignment.
- **Analytics Content Panels (Multi-Row/Multi-Column Grid):** Data visualization charts and system logs matching heights across grid tracks.

(b) Compare Flexbox and Grid based on layout needs.

Feature / Criteria	CSS Flexbox	CSS Grid Layout
Layout Dimension	One-Dimensional (Rows OR Columns).	Two-Dimensional (Rows AND Columns simultaneously).
Design Approach	Content-driven (items size based on inner content).	Layout-driven (content fits into defined tracks).
Grid Track Alignment	Requires complex item width percentages.	Native track control with <code>grid-template-columns</code> .
Dashboard Role	Micro-layouts (card content, button groups).	Macro-layouts (overall dashboard layout scaffold).

(c) Choose the most suitable layout technique and justify your choice.

Selected Strategy: Hybrid Approach (CSS Grid Macro Scaffold + CSS Flexbox Micro Alignment).

Justification: CSS Grid manages two-dimensional structural areas (sidebar, header, content grid) cleanly using `grid-template-areas`. Within individual widgets, CSS Flexbox handles one-dimensional alignment of icons, labels, and action buttons.

(d) Evaluate the scalability and responsiveness of the chosen approach.

Short Explanation:

CSS Grid uses `repeat(auto-fit, minmax(220px, 1fr))` to automatically adjust KPI widget counts per row according to available screen width without requiring extra media queries.

Complete Source Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Web Application Dashboard Layout</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family:
'Times New Roman', serif; }
    body { background-color: #f1f5f9; }

    /* Dashboard Grid Container */
    .dashboard-container {
      display: grid;
      grid-template-columns: 220px 1fr;
      grid-template-rows: 60px 1fr;
      grid-template-areas:
        "sidebar header"
        "sidebar main";
      min-height: 100vh;
    }

    .header { grid-area: header; background: #ffffff; padding: 0 20px;
display: flex; align-items: center; justify-content: space-between;
border-bottom: 1px solid #e2e8f0; }
    .sidebar { grid-area: sidebar; background: #0f172a; color: #ffffff;
padding: 20px; }
    .sidebar h2 { color: #38bdf8; font-size: 18px; margin-bottom:
20px; }
    .sidebar li { list-style: none; margin-bottom: 12px; }
    .sidebar a { color: #cbd5e1; text-decoration: none; }

    .main-content { grid-area: main; padding: 20px; }

    /* Scalable Responsive KPI Grid */
    .widgets-grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
      gap: 15px; margin-bottom: 20px;
    }

    .card { background: #ffffff; padding: 15px; border-radius: 6px;
box-shadow: 0 2px 4px rgba(0,0,0,0.05); }
    .card h4 { color: #64748b; font-size: 13px; }
    .card .val { font-size: 22px; font-weight: bold; color: #0f172a;
margin-top: 5px; }

    .panel-grid { display: grid; grid-template-columns: 2fr 1fr; gap:
15px; }
    .panel { background: #ffffff; padding: 15px; border-radius: 6px;

```

```

min-height: 150px; }

@media (max-width: 768px) {
  .dashboard-container { grid-template-columns: 1fr; grid-template-
rows: auto auto 1fr; grid-template-areas: "header" "sidebar" "main"; }
  .panel-grid { grid-template-columns: 1fr; }
}
</style>
</head>
<body>

<div class="dashboard-container">
  <header class="header">
    <h3>Admin Dashboard</h3>
    <div>User: Sameera Banu</div>
  </header>

  <aside class="sidebar">
    <h2>Analytics</h2>
    <ul>
      <li><a href="#">Overview</a></li>
      <li><a href="#">Sales</a></li>
      <li><a href="#">Settings</a></li>
    </ul>
  </aside>

  <main class="main-content">
    <div class="widgets-grid">
      <div class="card"><h4>Total Revenue</h4><div
class="val">$45,230</div></div>
      <div class="card"><h4>New Users</h4><div class="val">+1,205</
div></div>
      <div class="card"><h4>Active Sessions</h4><div
class="val">432</div></div>
      <div class="card"><h4>Conversion</h4><div class="val">3.2%</
div></div>
    </div>

    <div class="panel-grid">
      <div class="panel"><h4>Sales Performance Chart</h4><p
style="color:#64748b; font-size:12px; margin-top:10px;">[ Dynamic Graph
Output ]</p></div>
      <div class="panel"><h4>System Activity</h4><p
style="color:#64748b; font-size:12px; margin-top:10px;">[ Real-time
Activity Logs ]</p></div>
    </div>
  </main>
</div>

</body>
</html>

```

Output & Browser Screenshots:

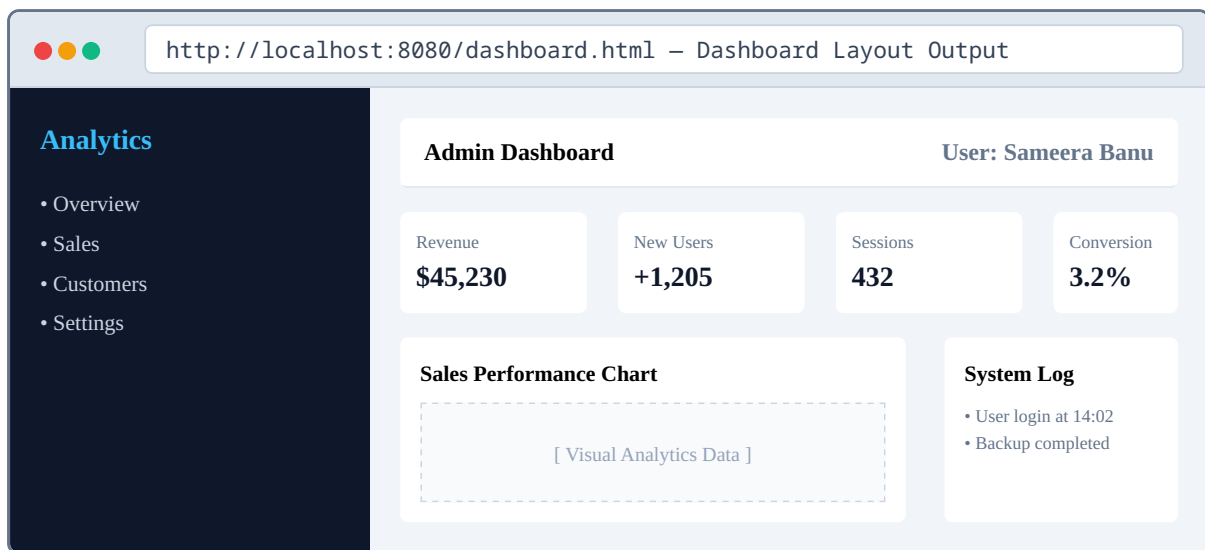


Figure 4: Dashboard Layout

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout.

A standard mobile-first blog architecture requires structural separation across four core sections:

- **Header & Navigation Banner:** Displays blog title, tagline, and navigation bar.
- **Main Article Container:** Contains the post headline, metadata (author, date), hero banner image, and structured body paragraphs.
- **Sidebar Panel:** Displays author biography, post categories, search input, and social sharing icons.
- **Footer:** Displays copyright notices, RSS links, and legal terms.

(b) Design a mobile-first layout using CSS.

(c) Demonstrate how the layout scales for larger screens using media queries.

Short Explanation:

The mobile-first workflow styles base elements for narrow mobile displays first without media queries. Progressive enhancement using `@media (min-width: 1024px)` expands the

layout into a two-column desktop grid where the article occupies 3 fr units and the sidebar occupies 1 fr unit.

Complete Source Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Mobile-First Blog Layout</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family:
'Times New Roman', serif; }
    body { background-color: #fafafa; color: #333333; line-height:
1.6; }

    .header { background-color: #2c3e50; color: #ffffff; padding: 20px;
text-align: center; }

    .container { width: 92%; margin: 20px auto; display: flex; flex-
direction: column; gap: 20px; }

    .main-article { background: #ffffff; padding: 18px; border-radius:
6px; box-shadow: 0 2px 5px rgba(0,0,0,0.05); }
    .hero-img { width: 100%; height: 180px; background-color: #cbd5e1;
border-radius: 4px; display: flex; align-items: center; justify-
content: center; color: #475569; margin-bottom: 15px; }

    .sidebar { background: #ffffff; padding: 18px; border-radius: 6px;
box-shadow: 0 2px 5px rgba(0,0,0,0.05); }

    .footer { background: #2c3e50; color: #ffffff; text-align: center;
padding: 12px; margin-top: 20px; font-size: 13px; }

    /* Desktop Scaling */
    @media (min-width: 1024px) {
      .container { display: grid; grid-template-columns: 3fr 1fr; max-
width: 1000px; }
    }
  </style>
</head>
<body>

  <header class="header">
    <h1>Tech Insights Blog</h1>
    <p style="font-size:13px;">Exploring Web Architecture & Modern
CSS</p>
  </header>

  <div class="container">
    <main class="main-article">
      <div class="hero-img">[ Featured Hero Image ]</div>
      <h2>Building Responsive Web Applications Mobile-First</h2>
      <p style="font-size:12px; color:#666; margin-bottom:10px;">By

```

```

Sameera Banu | Published July 2026</p>
  <p>Designing mobile-first guarantees optimized performance and
  clean visual layout structures across constrained viewports before
  scaling up to desktop displays...</p>
</main>

  <aside class="sidebar">
    <h3>About Author</h3>
    <p style="font-size:13px; margin-top:5px;">Sameera Banu. P – Web
Developer.</p>
    <h3 style="margin-top:15px;">Categories</h3>
    <ul style="font-size:13px; margin-left:15px; margin-top:5px;">
      <li>CSS Grid & Flexbox</li>
      <li>JavaScript Logic</li>
    </ul>
  </aside>
</div>

  <footer class="footer">
    <p>&copy; 2026 Tech Insights. All Rights Reserved.</p>
  </footer>

</body>
</html>

```

Output & Browser Screenshots:

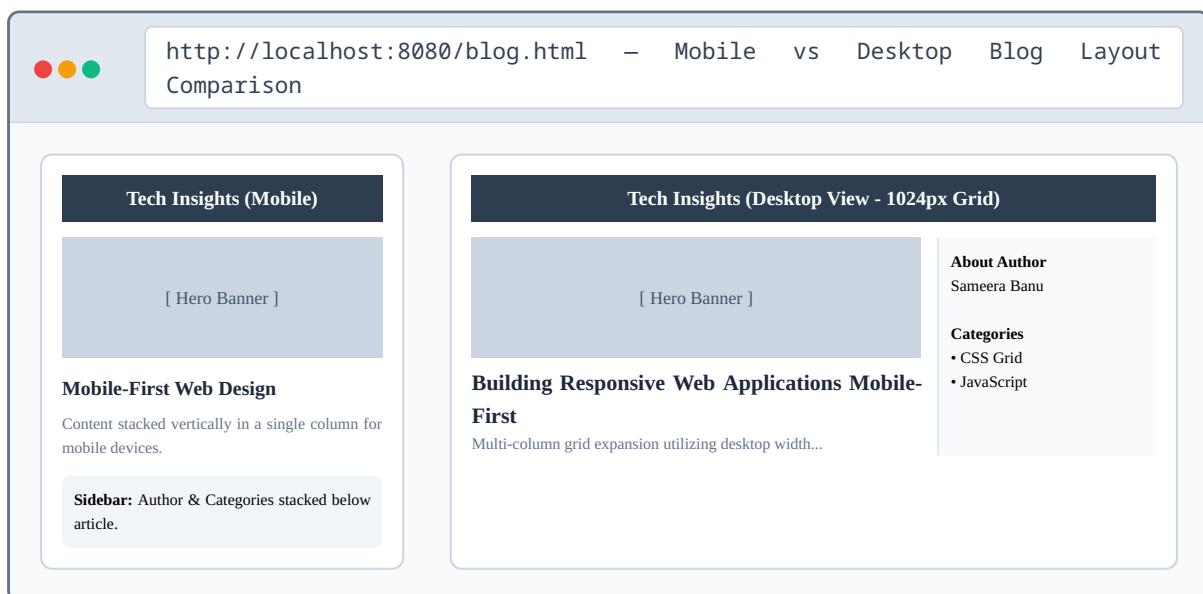


Figure 5: Blog Layout

(d) Evaluate accessibility and readability of the design.

Readability Evaluation: The layout restricts article max-width on wide displays, preserving an optimal reading line length of 60 to 75 characters per line. Line height is set to 1.6 for comfortable body paragraph scanning.

Accessibility Standards: Uses structural HTML5 landmarks (`<header>`, `<main>`, `<article>`, `<aside>`, `<footer>`) to allow screen readers to navigate content regions efficiently.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction.

To implement interactive client-side menus, the following DOM events are essential:

- `click` : Triggers dynamic toggling of dropdown visibility on touch and desktop screens.
- `mouseenter` / `mouseleave` : Handles hover-triggered dropdown expansion on desktop devices.
- `keydown` : Intercepts keyboard navigation (e.g., closing open dropdown menus when the `Escape` key is pressed).
- `focusout` / `blur` : Automatically closes open submenus when keyboard focus shifts away.

(b) Design event handling logic for menu toggle functionality.

(c) Implement interaction for user actions (click/hover).

Short Explanation:

The implementation combines mouse hover events (`mouseenter` / `mouseleave`) for desktop interactions with `click` event listeners for mobile devices and keyboard accessibility. An outside click handler closes active dropdowns when clicking elsewhere on the page.

Complete Source Code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Dynamic Interactive Menu</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; font-family:
'Times New Roman', serif; }
    body { background-color: #f8fafc; padding: 30px; }

    .menu-bar { background-color: #1e293b; display: flex; border-
radius: 6px; list-style: none; padding: 0 10px; }
    .menu-item { position: relative; }
    .menu-link { display: block; padding: 12px 18px; color: #ffffff;
text-decoration: none; cursor: pointer; }
    .menu-link:hover, .menu-item.show > .menu-link { background-color:
#334155; color: #38bdf8; }

    /* Submenu Panel */
    .submenu {
      display: none; position: absolute; top: 100%; left: 0;
      background-color: #334155; min-width: 160px;
      box-shadow: 0 4px 8px rgba(0,0,0,0.15); border-radius: 0 0 6px
6px; list-style: none;
    }
    .submenu a { display: block; padding: 10px 14px; color: #ffffff;
text-decoration: none; font-size: 13px; }
    .submenu a:hover { background-color: #475569; color: #38bdf8; }

    .menu-item.show .submenu { display: block; }
  </style>
</head>
<body>

  <ul class="menu-bar">
    <li class="menu-item"><a class="menu-link" href="#">Home</a></li>
    <li class="menu-item dropdown">
      <a class="menu-link" href="#">Services ▼</a>
      <ul class="submenu">
        <li><a href="#">Web Development</a></li>
        <li><a href="#">UI/UX Design</a></li>
        <li><a href="#">Cloud Hosting</a></li>
      </ul>
    </li>
    <li class="menu-item"><a class="menu-link" href="#">Contact</a></
li>
  </ul>

```

```

<script>
  document.addEventListener('DOMContentLoaded', () => {
    const dropdowns = document.querySelectorAll('.dropdown');

    dropdowns.forEach(dropdown => {
      // Toggle on click
      dropdown.addEventListener('click', function(e) {
        e.stopPropagation();
        dropdowns.forEach(item => { if(item !== dropdown)
item.classList.remove('show'); });
        this.classList.toggle('show');
      });

      // Hover support for desktop
      dropdown.addEventListener('mouseenter', function()
{ this.classList.add('show'); });
      dropdown.addEventListener('mouseleave', function()
{ this.classList.remove('show'); });
    });

    // Close on document click outside
    document.addEventListener('click', () => {
      dropdowns.forEach(item => item.classList.remove('show'));
    });

    // Close on Escape key press
    document.addEventListener('keydown', (e) => {
      if (e.key === 'Escape') dropdowns.forEach(item =>
item.classList.remove('show'));
    });
  });
</script>
</body>
</html>

```

Output & Browser Screenshots:

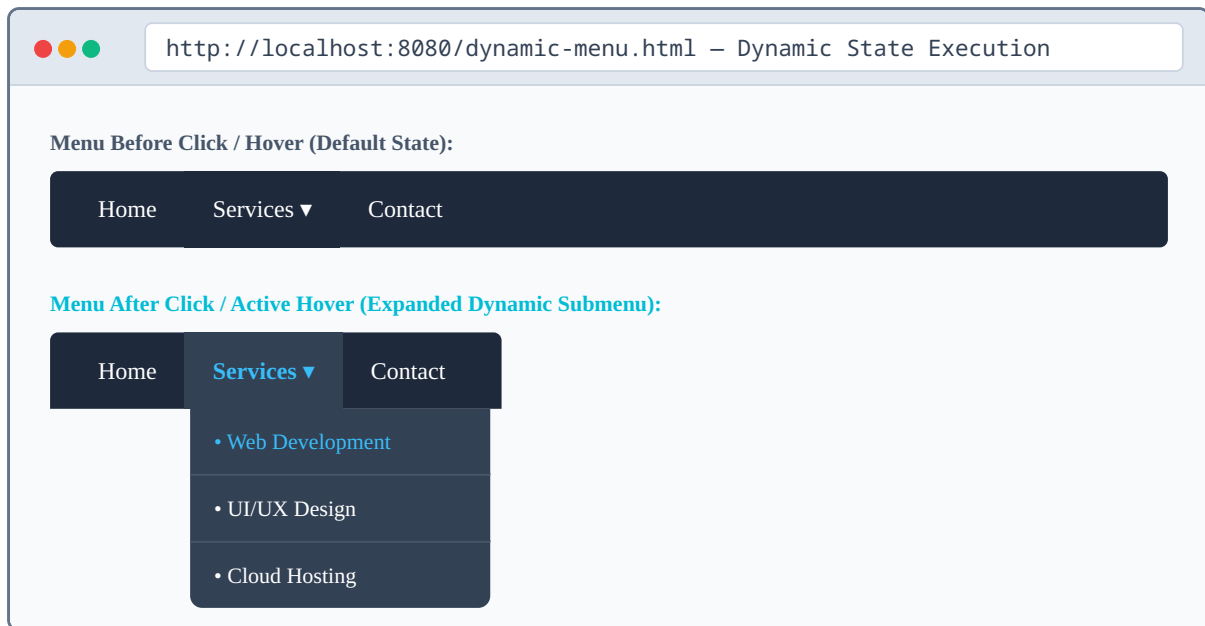


Figure 6: Dynamic Menu

(d) Evaluate performance and suggest improvements.

Performance Evaluation: Scoping event listeners to target dropdown containers prevents global listener overhead, minimizing CPU utilization during menu interaction.

Suggested Improvements:

- **Dynamic ARIA State Updates:** Dynamically update `aria-expanded="true/false"` on the trigger element when submenus open or close.
- **Keyboard Focus Trapping:** Manage keyboard tab focus within expanded dropdown menus to enhance accessibility for assistive technology users.

CODE OF CONDUCT

I **Sameera Banu. P / 192372172** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sameera Banu. P

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts.	Good understanding with minor gaps.	Basic understanding with some errors.	Poor understanding or incorrect concepts.
Design / Implementation	30	Well-structured, efficient, and responsive design implementation.	Correct implementation with minor issues.	Basic implementation with inconsistencies.	Incorrect or incomplete implementation.
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/ techniques.	Appropriate use of tools.	Limited or inconsistent use.	Incorrect or minimal use.
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization.	Good analysis with justification.	Basic analysis with limited depth.	Poor or no analysis.
Documentation / Clarity	10	Clear, well-structured, and properly presented solution.	Mostly clear with minor issues.	Basic clarity, missing details.	Poor or unclear documentation.